Git Issues Faced and Solutions – Phase 1.1 Project

Issue 1: All Folders Appearing in New Branch

Problem

After initializing Git in the `phase 1.1` folder, a new branch named `day4` was created. However, when pushing this branch, all folders (day1, day2, day3, day4) were included instead of only the `day4` folder.

Cause

Git branches store the entire repository snapshot, not individual folders. Since Git was initialized at the root (`phase 1.1`), every branch contained all folders.for example

- A branch `day3` → should contain only `day3` folder
- A branch `day4` → should contain only `day4` folder
- A branch `jwt_auth` → should contain only `jwt_authentication` folder
- A branch `alembic` → should contain only `alembic` folder

When pushing a branch, wanted **only that folder to go**, not the whole project.

Solution

Single Repo with Separate Branches

- Keep `main` branch empty
- Create new branches from `main`
- Add only the required folder in each branch
- Commit and push

Advanced method used:

git checkout --orphan day4

git rm -rf .
git add day4
git commit -m "Day4 only branch"
git push -u origin day4

This ensures each branch contains only one folder.

Issue 2: Unable to Delete a Branch

Problem

Attempted to delete a branch but received an error because:

- The branch was currently checked out
- Or it was not fully merged

Solution

Step 1: Switch to another branch

git checkout main

Step 2: Delete branch safely

git branch -d day4

Step 3: Force delete (if not merged)

git branch -D day4

Step 4: Delete remote branch (if pushed)

git push origin --delete day4

Problem: Deleting files removed them from PC

You did something like:

git rm -r .

or cleaned branch

And your folder (like jwt_authentication) disappeared from PC.

Why?

Because:

- Git deletes files from working directory

Issue :alembic folder

Today, I faced an issue while working with Git branches. The `alembic` folder appeared empty when switching branches, and Git prevented me from moving to the `main` branch because it reported that untracked files would be overwritten. Additionally, the folder was originally added as a submodule, which caused Git to treat it differently from a normal folder.

How the Issue Was Solved

First, I identified that the `alembic` folder was configured as a Git submodule. I removed the submodule reference using `git rm --cached alembic` and deleted its configuration from `.git/config`. Then, I re-added the folder as a normal tracked directory and committed the changes. After converting it to a standard folder, I was able to switch branches successfully and create a dedicated `alembic` branch containing all required files without errors.

Step 1 — Remove submodule reference from Git

git rm --cached alembic

> This removes the alembic folder from Git tracking **without deleting files**.

Step 2 — Delete leftover submodule files

Remove the `.gitmodules` entry for alembic:

1. Open `.gitmodules` in your repo root
2. Delete the section for alembic (if it exists), e.g.:

```
[submodule "alembic"]
  path = alembic
  url = ...
```

3. Save the file

Then remove the leftover submodule config in `.git/config`

Step 3 — Re-add alembic folder as a normal folder

```
git add alembic
git commit -m "Convert alembic from submodule to normal folder"
```

Now Git will track all files inside `alembic`.

Step 4 — Create alembic branch (if not done yet)

```
git checkout -b alembic
```

Step 5 — Remove other folders in this branch

```
git rm -r day1 day2 day3 day4 jwt_authentication
git commit -m "Keep only alembic folder"
```

Step 6 — Push branch

```
git push origin alembic
```

✅ Now your `alembic` branch will have the **full alembic folder with all files**, nothing else.

Issue 3: Major Folder Appeared Deleted from Laptop

Problem

After running a Git command (such as `git rm -r *` while cleaning a branch), the main project folder appeared to be deleted from the laptop. It looked like all files were removed.

Cause

The command removed files from the current branch's working directory. However, the files were not permanently deleted from Git history. They still existed in another branch (e.g., `main`).

Git changes the working directory based on the branch currently checked out.

Solution

The issue was resolved by switching back to the `main` branch:

git checkout main

After switching branches, all files were automatically restored because they existed in the `main` branch snapshot.